

SQL

이 경 미

- SQL 개요
- 데이터 정의 언어 DDL
 - 데이터 베이스 생성과 삭제
 - 테이블 생성
 - 테이블 변경
 - 테이블 삭제

SQL 개요

■ 데이터베이스 질의어

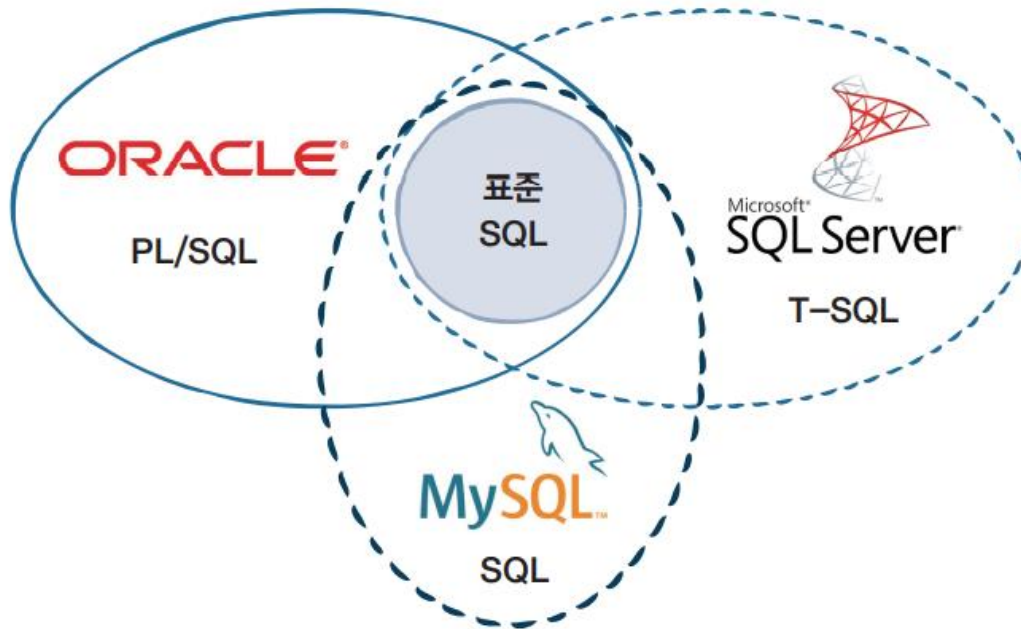
- 데이터베이스를 구축하기 위한 도구로서 현대 정보시스템에서 매우 중요한 역할 차지
- 질의어(Query Language)는 **검색 언어**라는 의미지만
- 데이터를 **검색**하는 역할 외에 **데이터의 입력, 수정, 삭제, 병행 제어, 복구** 등 다양한 기능을 제공하는 종합적인 언어

■ SQL (Structured Query Language)

- 관계형 데이터베이스 관리 시스템(RDBMS)의 데이터를 관리하기 위해 설계된 특수 목적의 프로그래밍 언어
- ‘에스큐엘’ 또는 ‘시퀄’로 읽음
- 1970년대 말 미국 IBM사의 한 연구소에서 데이터베이스 유형에 관계없이 사용자들이 편리하게 사용할 수 있는 질의어를 만들기 위해 개발
- 초기에는 SEQUEL(Structured English Query Language, 구조 영어 질의어)이라는 이름으로 시작
- 많은 수의 데이터베이스 관련 프로그램들이 SQL을 표준으로 채택하면서 자신의 제품에 특화 시킨 SQL 사용

SQL 개요

- DBMS 제품별 특화된 SQL



이미지출처. 혼자공부하는 SQL, 우재남, 한빛미디어

SQL 개요

■ SQL 문의 기능별 분류

■ 데이터 정의어 (DDL)

- 테이블이나 관계의 구조를 생성하는 데 사용
- 스키마, 도메인, 테이블, 뷰, 인덱스를 정의/변경/제거
- **CREATE, ALTER, DROP** 문

■ 데이터 조작어 (DML)

- 테이블의 데이터를 검색, 삽입, 수정, 삭제하는 데 사용
- **SELECT, INSERT, DELETE, UPDATE** 문

■ 데이터 제어어 (DCL)

- 데이터의 사용 권한을 관리하는 데 사용
- 데이터베이스 트랜잭션 명시(완료/취소)
- 권한 부여/취소
- **GRANT, REVOKE, COMMIT, ROLLBACK** 문

SQL : 데이터 정의어(DDL)

데이터 정의어(DDL)

- **CREATE문**
 - 데이터베이스 및 객체 생성
- **ALTER문**
 - 데이터베이스 객체 변경
- **DROP 문**
 - 데이터베이스 및 객체 삭제

데이터 정의어(DDL)

- CREATE 문

- 테이블, 도메인, 뷰, 인덱스, 스키마 구조 정의
- CREATE TABLE
- CREATE SCHEMA
- CREATE DATABASE
- CREATE DOMAIN
- CREATE INDEX
- CREATE VIEW

데이터베이스 생성/삭제

■ 데이터베이스(스키마) 생성

CREATE DATABASE 데이터베이스명 [**DEFAULT CHARACTER SET** 인코딩];

CREATE SCHEMA 데이터베이스명 [**DEFAULT CHARACTER SET** 인코딩];

```
-- 스키마(데이터베이스 생성)
```

```
CREATE SCHEMA sqlldb3 DEFAULT CHARACTER SET utf8;
```

```
CREATE DATABASE sqlldb4 DEFAULT CHARACTER SET UTF8MB4;
```

Utf8로 사용할 경우 경고 (UTF8MB4 in a future release)

참고. 대소문자 구분 주의

- 데이터베이스이름, 테이블이름은 DBMS에 따라 대소문자 구분 여부가 다름
- MySQL의 경우 운영체제에 따라 대소문자 구분 여부 다름
 - 윈도우는 제한이 없으나, 유닉스, 리눅스, Mac에서는 대소문자 구분

데이터베이스 생성/삭제

■ 데이터베이스 삭제

DROP DATABASE 데이터베이스명

DROP SCHEMA 데이터베이스명

```
-- 스키마(데이터베이스 삭제)  
DROP SCHEMA sqldb3;  
DROP DATABASE sqldb4;
```

테이블 생성

■ CREATE TABLE

- 테이블 구성
 - 속성(열)과 속성(열)에 관한 제약 정의
 - 기본키(PRIMARY KEY), 외래키(FOREIGN KEY) 정의
-
- 기본 형식

CREATE TABLE 테이블명 (열이름 데이터타입 [, 제약조건]);

```
CREATE TABLE 테이블명 (  
    열이름 데이터타입  
    [NOT NULL] [UNIQUE] [DEFAULT 기본값] [CHECK 체크조건]  
    [, PRIMARY KEY 열이름]  
    [, FOREIGN KEY 열이름 REFERENCES 테이블명(열이름)]  
    [, CONSTRAINT명 CHECK(조건식)]  
);
```

테이블 생성

■ CREATE TABLE

■ 제약조건

표기형식	설명
NOT NULL	빈 값 허용하지 않음
DEFAULT	기본값으로 설정
PRIMARY KEY	기본키 설정
FOREIGN KEY	외래키 설정
REFERENCES	외래키가 참조할 테이블(부모 테이블) 설정
UNIQUE	중복값이 없도록 설정 (대체키 설정 의미)
CHECK	특정 내용의 제약 조건 설정
ON DELETE/ON UPDATE	참조되는 테이블의 행 삭제/갱신 시 옵션

참고. NULL

- 어떤 값도 정의되어 있지 않음을 의미함
- NULL 값을 허용한 경우 테이블 생성 시 NULL이 들어 감
- NULL 값을 허용하지 않는 경우 NULL 삽입 시 오류 발생

연습문제: 테이블 생성

■ 학생(student)과 학과(department) 테이블 생성하고 데이터 입력(각 3개씩)

■ Testdb2 스키마에 생성

■ 제약조건

- 기본키 설정
- 학생은 학과에 소속
- 학생 이름과 학과 이름은 NULL 허용하지 않음
- 학년은 4를 기본값으로, 범위를 1 ~ 4로 설정
 - (AND 키워드 사용)

학과	학생
<u>학과코드</u>	<u>학번</u>
학과명	성명
전화번호	학년
	주소
	생년월일
	학과코드

	dptNo	dptName	dptTel
	1	컴퓨터...	02-1111-1111
	2	정보통...	031-2222-2222
▶	3	경영학과	032-3333-3333

	stdNo	stdName	stdYear	stdAddress	stdBirthDay	dptNo
	2018	홍길동	4	서울시 종로구	1998-01-01	1
	2021	이몽룡	1	경기 안양	2001-12-12	2
✎	2020	성춘향	2	전북 남원	2001-10-10	3

연습문제: 테이블 생성

- 교수, 과목, 성적 테이블을 추가하시오.

교수	과목	성적
<u>교수번호</u>	<u>과목코드</u>	<u>학번</u>
교수명	과목명	<u>과목코드</u>
직위	학점수	성적
전화번호	교수번호	등급
학과코드		

테이블 변경

■ ALTER 문

- 테이블 수정
- 테이블에 대한 정의 변경
- 새로운 열 추가, 특정 열의 디폴트값 변경, 특정 열 삭제 등 수행

■ ALTER 문의 기본 형식

ALTER TABLE

ADD : 열 추가

RENAME COLUMN : 열의 이름 변경

MODIFY : 열의 데이터 형식 변경

CHANGE : 열의 이름과 데이터 형식 변경

DROP COLUMN : 열 삭제

DROP : 여러 개의 열 삭제

DROP PRIMARY KEY : 기본키 삭제

DROP CONSTRAINT : 제약조건 삭제

연습문제: 테이블 변경(ALTER)

- product 테이블에 숫자 값을 갖는 prdStock과 제조일을 나타내는 prdDate 열 추가
- Product 테이블의 prdCompany 열을 NOT NULL로 변경
- publisher 테이블에 pubPhone, pubAddress 열 추가
- publisher 테이블에서 pubPhone 열 삭제

한 문장씩 실행
테이블 새로고침하고 결과 확인

테이블 삭제

■ 테이블 삭제

- **DROP TABLE** [PURGE | CASCADE CONSTRAINTS];
- 스키마, 도메인, 테이블, 뷰, 인덱스를 삭제하는 명령문
- 테이블 구조와 데이터 모두 삭제
- (데이터만 삭제 시 DELETE 문 사용 - DML)

SQL : DML

이 경 미

■ 데이터 조작 언어 DML

- 데이터 삽입 INSERT 문
- 데이터 갱신 UPDATE 문
- 데이터 삭제 DELETE 문
- 데이터 검색 SELECT 문
- 데이터 조건 검색 SELECT ~ WHERE 문
 - IN, NOT IN, IS NULL, IS NOT NULL, LIKE, BETWEEN
- 데이터 정렬 SELECT ~ ORDER BY 문
 - LIMIT
- 데이터 집계
 - SUM(), AVG(), MIN(), MAX(), COUNT(), COUNT(DISTINCT)
- 그룹별 집계 SELECT ~ GROUP BY 문
 - ROLLUP / HAVING

실습 데이터베이스 준비(1)

데이터베이스 준비

- 실습. 인터넷 쇼핑몰을 위한 데이터베이스 구축
 - Workbench를 사용
- 데이터베이스(스키마) 생성
 - DB 이름 : marketdb
- 테이블 생성
 - 고객 테이블 customer
 - 고객 회원타입 테이블 membertype
 - 상품 테이블 product
 - 주문 테이블 productorder

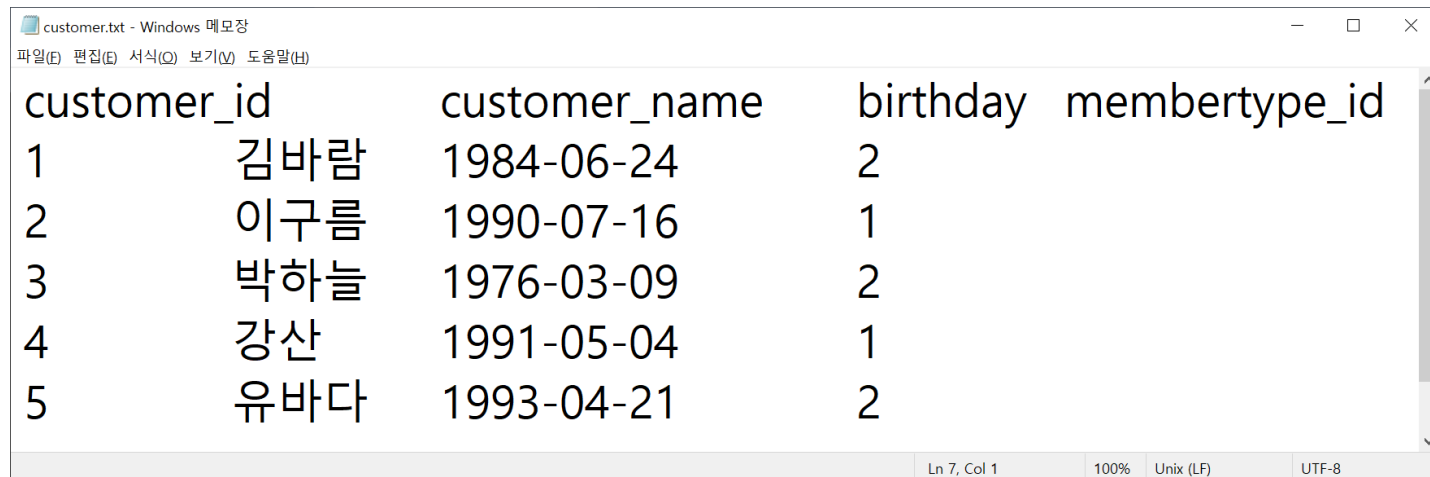
데이터베이스 준비

■ 테이블

■ 고객테이블 customer

- 고객의 정보 : 이름, 생년월일, 고객회원타입(회원/비회원), 고객ID

Column Name	Data type	PK/FK	NULL	의미
<u>customr_id</u>	INT	PK	No	고객 ID
customer_name	VARCHAR(45)		Yes	고객 이름
birthday	DATE		Yes	고객 생년월일
membertype_id	INT	FK	No	고객 회원타입



customer_id	customer_name	birthday	membertype_id
1	김바람	1984-06-24	2
2	이구름	1990-07-16	1
3	박하늘	1976-03-09	2
4	강산	1991-05-04	1
5	유바다	1993-04-21	2

데이터베이스 준비

■ 테이블

- 고객 회원타입 테이블 membertype
 - 보통회원/할인회원

Column Name	Data type	PK/FK	NULL	의미
<u>memberof id</u>	INT	PK	No	회원타입ID
memberof	VARCHAR(5)		No	회원타입명

```
memberof_id  memberof
1           보통 회원
2           할인 회원
```

학습용 데이터베이스 준비

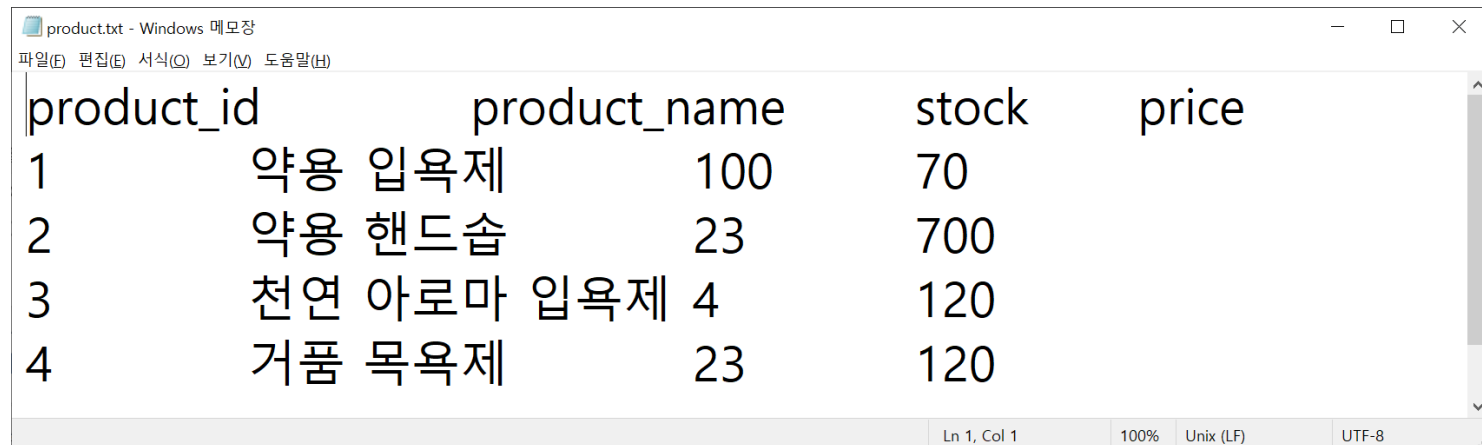
■ 테이블

■ 상품테이블 product

- 쇼핑몰에서 판매하는 상품정보 : ID, 상품명, 재고 수, 단가

Column Name	Data type	PK/FK	NULL	의미
<u>product_id</u>	INT	PK	No	상품 ID
product_name	VARCHAR(20)		No	상품명
stock	INT		Yes	재고 수
price	INT		Yes	단가

- stock과 price는 기본값을 0으로 하고 0이상의 값이 저장되게 제약조건을 둬



product_id	product_name	stock	price
1	약용 입욕제	100	70
2	약용 핸드숍	23	700
3	천연 아로마 입욕제	4	120
4	거품 목욕제	23	120

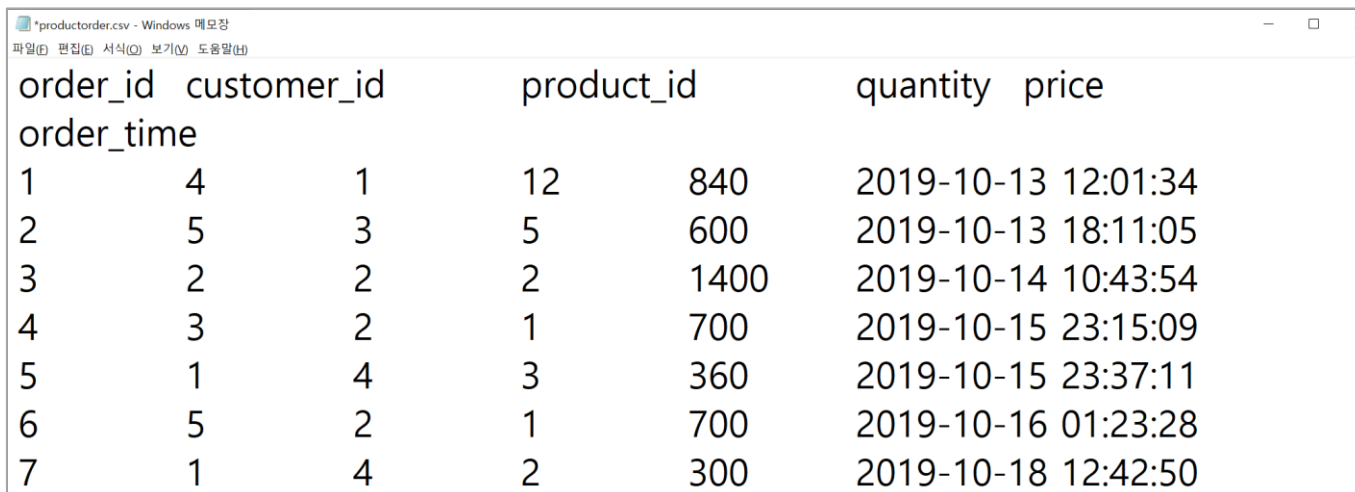
학습용 데이터베이스 준비

■ 테이블

■ 주문 테이블 productorder

Column Name	Data type	PK/FK	NULL	의미
<u>order_id</u>	INT	PK	No	주문 ID
customer_id	INT	FK	No	고객 ID
product_id	INT	FK	No	상품 ID
quantity	INT		No	주문 수
price	INT		No	금액
order_time	DATETIME		Yes	주문 일시

- order_id는 자동으로 증가되게 설정



order_id	customer_id	product_id	quantity	price	order_time
1	4	1	12	840	2019-10-13 12:01:34
2	5	3	5	600	2019-10-13 18:11:05
3	2	2	2	1400	2019-10-14 10:43:54
4	3	2	1	700	2019-10-15 23:15:09
5	1	4	3	360	2019-10-15 23:37:11
6	5	2	1	700	2019-10-16 01:23:28
7	1	4	2	300	2019-10-18 12:42:50

DML(Data Manipulation Language)

- 데이터 조작용어

- 데이터 입력 / 삭제 / 수정 / 검색
- INSERT 문
- DELETE 문
- UPDATE 문
- SELECT 문

데이터 삽입 INSERT

데이터 삽입 INSERT

■ INSERT 문

- 테이블에 새로운 행을 삽입하는 명령어

■ INSERT 문 기본 형식

INSERT INTO 테이블명 (열이름리스트) **VALUES** (값리스트);

■ INSERT 문 예

-- student 테이블에 행 삽입

```
INSERT INTO student (stdNo, stdName, stdYear, dptNo)
VALUES ('2018005', '변학도', 4, '1');
```

문자 : '1'

숫자 : 1

데이터 삽입 INSERT

■ 연습문제.

- INSERT 문을 사용하여 학과 / 학생 테이블에 다음과 같이 데이터 삽입하고 SELECT 문으로 조회

	dptNo	dptName	dptTel
▶	1	컴퓨터학과	02-1111-1111
	2	경영학과	02-2222-2222
	3	수학과	02-7777-7777

	stdNo	stdName	stdYear	stdAddress	stdBirthDay	dptNo
▶	2018002	이몽룡	4	서울시 강남구	1998-05-07	1
	2019003	홍길동	3	경기도 안양시	1999-11-11	2
	2021003	성춘향	1	전라북도 남원시	2002-01-02	3
	2021004	변학도	1	서울시 종로구	2000-11-11	2

데이터 갱신 UPDATE

데이터 갱신 UPDATE

■ UPDATE 문

- 특정 열의 값을 수정하는 명령어
- 조건에 맞는 행을 찾아서 열의 값 수정

■ UPDATE 문 기본 형식

UPDATE 테이블명 **SET** 열=값 **WHERE** 조건;

■ UPDATE 문 예

--상품번호가 5인 행의 상품명을 'UHD TV'로 수정

UPDATE product **SET** prdName='UHD TV' **WHERE** prdNo='5';

데이터 삭제 DELETE

데이터 삭제 DELETE

- DELETE 문

- 테이블에 있는 기존 행을 삭제하는 명령어

- DELETE 문 기본 형식

DELETE FROM 테이블명 **WHERE** 조건;

- DELETE 문 예

--상품명이 '그늘막 텐트'인 행 삭제

DELETE FROM product **WHERE** prdName = '그늘막 텐트';

--테이블의 모든 행 삭제

DELETE FROM product;

데이터 삭제 DELETE

■ 연습문제

- book 테이블에 다음과 같이 행 삽입
(출판사 데이터는 테이블 구조에 맞게 입력)

도서번호	도서명	가격	발행일	출판사
9	JAVA 프로그래밍	30000	2022-03-10	서울 출판사
10	파이썬 데이터 과학	24000	2021-02-05	강남 출판사

- book 테이블에서 도서명이 ' 자바'인 행의 가격을 22000으로 변경
- book 테이블에서 발행일이 2021년도인 행 삭제

종합문제

■ 다음과 같이 SQL 문 작성

1. **고객** 테이블 (**customer**) 생성
2. **고객** 테이블의 **전화번호** 열을 **NOT NULL**로 변경
3. **고객** 테이블에 '**성별**', '**나이**' 열 추가
4. **고객** 테이블에 데이터 **삽입** (3개)
5. 고객명이 홍길동인 고객의 전화번호 값 수정 (값은 임의로)
6. 나이가 20살 미만인 고객 삭제

데이터 검색 SELECT

데이터 검색 SELECT

- SELECT 문
 - 테이블에서 조건에 맞는 행 검색
- SELECT 문 기본 형식

SELECT [ALL | DISTINCT] 열이름 리스트
FROM 테이블명

순서 ① [WHERE 검색조건(들)]
② [GROUP BY 열이름]
③ [HAVING 검색조건(들)]
④ [ORDER BY 열이름 [ASC | DESC]]

데이터 검색 SELECT

■ SELECT 문 기본 형식

기능	설명
SELECT 열이름	검색할 열 기술
FROM 테이블명	데이터를 검색할 테이블명 기술
WHERE 조건	질의 결과에 포함될 행들이 만족해야 할 조건 기술
ORDER BY 열이름	특정 열 의 값을 기준으로 질의 결과 정렬 ASC : 오름차순 DESC : 내림차순
GROUP BY 열이름	그룹 질의를 기술할 때 사용 특정 열 로 그룹화한 후 각 그룹에 대해 한 행씩 질의 결과 생성
HAVING 조건	GROUP BY 절에 의해 구성된 그룹들에 대해 적용할 조건 기술

데이터 검색 SELECT

■ SELECT 문 실행 순서

SELECT [ALL | DISTINCT] **열이름 리스트**
FROM **테이블명**
WHERE 검색조건(들)
GROUP BY **열이름**
HAVING 검색조건(들)
ORDER BY **열이름** [ASC | DESC]

SELECT

5

DISTINCT

6

FROM

1

WHERE

2

GROUP BY

3

HAVING

4

ORDER BY

7

데이터 검색 SELECT

■ 예.

--도서(book) 테이블에서 모든 행 검색하여 반환

--모든(*) 열 포함

```
SELECT * FROM book;
```

-- 도서(book) 테이블에서 모든 행을 검색하여 도서명과 가격만 반환

```
SELECT bookName, bookPrice FROM book;
```

-- 도서(book) 테이블에서 가격이 30000이상인 행을 검색하여

-- 도서명과 가격만 반환

```
SELECT bookName, bookPrice FROM book  
WHERE bookPrice >= 30000; --조건
```

데이터 검색 SELECT DISTINCT

■ 중복제거

- * : 모든 열 출력
- **DISTINCT**
 - 속성값이 중복되는 것이 있으면 한 번만 출력
 - 예 :
 - 주문 테이블에서 '홍길동'이 여러 번 주문한 경우
 - 한번이라도 주문한적이 있는 고객명 리스트 출력 시 DISTINCT 사용하면 '홍길동' 한 번만 출력

데이터 검색 SELECT DISTINCT

■ 실습

- 스키마 생성 : sqldb2
- 테이블 생성
 - csv 파일의 데이터 импорт
- 테이블 데이터 타입 변경
 - text 는 VARCHAR()으로
 - 모든 no는 문자 타입으로
 - 날짜는 DATE 타입으로
- 기본키/외래키 설정

publisher		book		client		bookSale	
<u>pubNo</u>		<u>bookNo</u>		<u>clientNo</u>		<u>bsNo</u>	
pubName		bookName		clientName		bsDate	
		bookAuthor		clientPhone		bsQty	
		bookPrice		clientAddress		clientNo	
		bookDate		clientBirth		bookNo	
		bookStock		clientHobby			
		pubNo		clientGender			

데이터 조건 검색

데이터 조건검색 SELECT ... WHERE

■ WHERE 조건

- 비교 (=, <, >, <=, >=, !=)
- 범위 (BETWEEN)
- 리스트에 포함 (IN, NOT IN)
- NULL (IS NULL, IS NOT NULL)
- 논리(AND, OR)
- 패턴 매칭 (LIKE)

데이터 조건검색 SELECT ... WHERE

■ WHERE 조건

종류	연산자	설명/예
비교	=, <, >, <=, >=, !=	값의 크기를 비교하여 질의 검색 price >= 10000
범위	BETWEEN	검사 값이 기술된 두 값 사이에 속하는지 검사하여 질의 처리 price BETWEEN 10000 AND 20000
리스트에 포함	IN, NOT IN	검사 값이 주어진 값의 리스트에 속하는지 여부를 검사 price IN (10000, 20000, 30000)
NULL	IS NULL, IS NOT NULL	price IS NULL
논리	AND, OR	단순 탐색 조건을 결합하여 질의 처리 (price <= 10000) AND (name LIKE '김%')
패턴 매칭 문자열 검색	LIKE	문자열의 일부가 일치하는 데이터 검색 prdName LIKE '%프린터%'

데이터 조건검색 SELECT ... WHERE

■ WHERE 조건 : 패턴 매칭 (LIKE)

문자열 연산 예	설명
'홍%'	'홍'으로 시작하는 문자열 검색
'%길%'	'길'을 포함하는 문자열 ('길' 앞,뒤에 아무 문자나 와도 됨)
'%동'	'동'으로 끝나는 문자열
' _ '	4개의 문자로 구성된 문자열(밑줄문자 1개가 1개 문자)

와일드카드 문자	설명	예
%	0개 이상의 문자를 가진 문자열	LIKE '홍%'
_	단일 문자(수 만큼의 문자로 구성)	LIKE ' _ '

데이터 정렬

데이터 정렬 SELECT ... ORDER BY

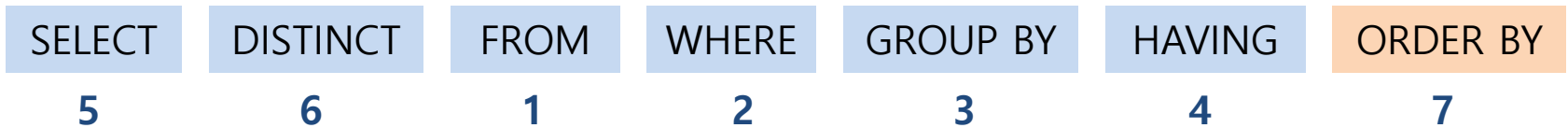
■ ORDER BY

- 특정 열의 값을 기준으로 질의 결과 정렬
- ASC : 오름차순 (생략 가능)
- DESC : 내림차순

■ ORDER BY 구는 가장 마지막에 실행됨

- SELECT 절 뒤에 수행됨

실행 순서



데이터 집계

데이터 집계 SELECT

■ 집계 함수

- SUM() : 합계
- AVG() : 평균
- COUNT() : 선택된 열의 행 수(널 값은 제외)
- COUNT(*) : 전체 행의 수
- MAX() : 최대
- MIN() : 최소

■ 집계함수를 적는 곳은 정해져 있다

- SELECT 구, HAVING 구, ORDER BY 구

■ 집계함수는 1개의 값을 반환하는 것만 함께 적을 수 있다

- 상수, 집계함수, DISTINCT, 연산자 등

그룹별 집계

그룹별 집계 SELECT ... GROUP BY

■ GROUP BY <속성>

- 그룹 질의를 기술할 때 사용
- 특정 열로 그룹화한 후 각 그룹에 대해 한 행씩 질의 결과 생성
- 주의!!
 - SELECT 구에는 1개의 값을 반환하는 상수, GROUP BY에서 사용한 <열>, 집계 함수만 나올 수 있음

```
SELECT bookNo, SUM(bsQty) AS "주문량 합계"  
FROM bookSale  
GROUP BY bookNo;
```

- GROUP BY에서 사용하는 컬럼을 '집약키'라고 부름
- 집약키는 여러 개 지정할 수 있다

그룹별 집계 SELECT ... GROUP BY

■ 연습문제

1. 도서 테이블에서 가격 순으로 내림차순 정렬하여, 도서명, 저자, 가격 출력 (가격이 같으면 저자 순으로 오름차순 정렬)
2. 도서 테이블에서 저자에 '길동'이 들어가는 도서의 총 재고 수량 계산하여 출력
3. 도서 테이블에서 '서울 출판사' 도서 중 최고가와 최저가 출력
4. 도서 테이블에서 출판사별로 총 재고수량과 평균 재고 수량 계산하여 출력 ('총 재고 수량'으로 내림차순 정렬)
5. 도서판매 테이블에서 고객별로 '총 주문 수량'과 '총 주문 건수' 출력. 단 주문 건수가 2이상인 고객만 해당

정리

DDL / DML 정리 (1)

- DDL (Data Definition Language, 데이터 정의어)
 - CREATE / ALTER / DROP
 - 데이터베이스, 테이블, 뷰, 인덱스 등의 데이터베이스 개체를 생성/삭제/변경하는 역할
 - 직접 데이터베이스의 테이블에 영향을 미치기 때문에
 - DDL 명령어를 입력하는 순간 처리 작업이 즉시 완료 (AUTO COMMIT)
 - 따라서 작업 취소(ROLLBACK)이나 완료(COMMIT)을 시킬 없음

DDL / DML 정리 (2)

- DML (Data Manipulation Language, 데이터 조작어)
 - INSERT / UPDATE / DELETE / SELECT
 - 데이터를 삽입/수정/삭제/조회 하는데 사용
 - 대상은 테이블의 행
 - 조작 작업이 메모리 버퍼에서 수행되어 실시간으로 테이블에 영향을 미치지 않음
(실수가 있었을 경우 작업 취소 가능) - 트랜잭션 발생
 - DML 명령어가 실제 테이블에 반영되게 하기 위해서는 COMMIT 명령어를 입력하여 작업을 종료해야 함

DDL / DML 정리 (3)

■ 트랜잭션 (Transaction)

- DBMS에서 데이터를 다루는 논리적인 작업의 단위
- 데이터베이스 관리 시스템에서 회복 및 병행 수행 시 처리되는 작업의 논리적 단위로 사용
- 하나의 트랜잭션은 정상적으로 종료될 경우 COMMIT 연산이 수행되고
- 비정상적으로 종료될 경우 ROLLBACK 연산 수행해서 작업을 취소

DDL / DML 정리 (4)

■ COMMIT 연산

- 트랜잭션 처리가 정상적으로 종료되어
- 트랜잭션이 수행한 변경 내용을 데이터베이스에 반영하는 연산
- 내용을 변경한 트랜잭션이 완료되면 그 트랜잭션에 의해 데이터베이스는 새롭게 일관된 상태로 변경되며
- 이 상태는 시스템에 오류가 발생되더라도 취소되지 않음

■ ROLLBACK 연산

- 하나의 트랜잭션 처리가 비정상적으로 종료되어
- 데이터베이스의 일관성이 깨졌을 때 트랜잭션이 행한 모든 변경 작업을 취소하고 이전 상태로 되돌리는 연산

SQL : DML (조인과 서브쿼리)

이 경 미

학습 내용

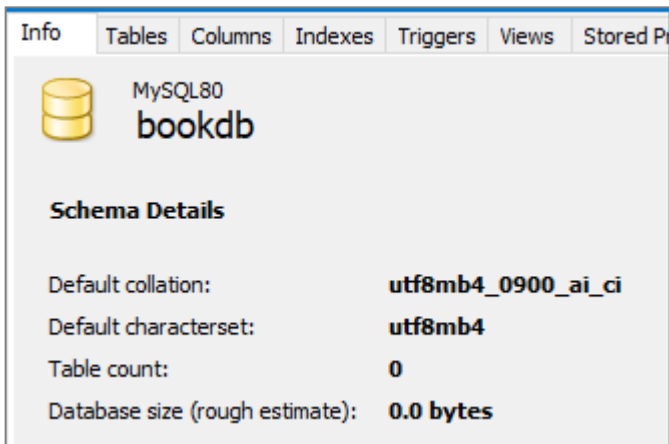
- 실습 데이터베이스 준비
- 조인
- 서브 쿼리
- 내장 함수

실습 데이터베이스 준비(2)

실습 데이터베이스 준비

■ 스키마(데이터베이스) 생성 : bookdb

```
-- book 데이터베이스 생성  
drop database if exists bookdb;  
create database bookdb;
```



실습 데이터베이스 준비

■ 테이블 생성

- CSV 파일의 데이터 импорт

publisher

<u>pubNo</u>
pubName

book

<u>bookNo</u>
bookName
bookAuthor
bookPrice
bookDate
bookStock
pubNo

bookClient

<u>clientNo</u>
clientName
clientPhone
clientAddress
clientBirth
clientHobby
clientGender

bookSale

<u>bsNo</u>
bsDate
bsQty
clientNo
bookNo

■ 테이블 데이터 타입 변경

- text 는 VARCHAR()으로
- 모든 no는 문자 타입으로
- 날짜는 DATE 타입으로

■ 기본키/외래키 설정

실습 데이터베이스 준비

■ 출판사테이블 publisher

Column Name	Data type	PK/FK	NULL	의미
<u>pubNo</u>	VARCHAR(10)	PK	No	출판사No
pubName	VARCHAR(20)		Yes	출판사명

pubNo	pubName
1	서울 출판사
2	도서출판 강남
3	정보출판사

실습 데이터베이스 준비

■ 도서테이블 book

Column Name	Data type	PK/FK	NULL	의미
<u>bookNo</u>	VARCHAR(10)	PK	No	도서No
bookName	VARCHAR(20)		Yes	도서명
bookAuthor	VARCHAR(30)		Yes	저자명
bookPrice	INT		Yes	도서가격
bookDate	DATE		Yes	출판일
bookStock	INT		Yes	재고량
pubNo	VARCHAR(10)	FK	No	출판사No

bookNo	bookName	bookAuthor	bookPrice	bookDate	bookStock	pubNo
1001	데이터베이스 이론	홍길동	22000	2018-11-11	5	3
1002	자바 프로그래밍	이몽룡	25000	2020-12-12	4	1
1003	인터넷프로그래밍	성춘향	30000	2019-02-10	10	2
1004	안드로이드 프로그래밍	변학도	23000	2017-10-10	2	1
1005	안드로이드 앱	강길동	26000	2019-01-11	5	2
1006	MS SQL SERVER 2014	박지성	35000	2019-03-25	7	3
1007	HTML & CSS	손흥민	18000	2021-01-30	3	1
1008	MFC 입문	류현진	20000	2014-12-12	5	1
1009	안드로이드 게임 제작	나길동	33000	2017-10-31	5	2
1010	C++ 객체지향 프로그래밍	추신수	24000	2018-04-20	2	3
1011	JSP 웹 프로그래밍	김연아	27000	2019-10-23	8	1

실습 데이터베이스 준비

■ 고객테이블 bookClient

Column Name	Data type	PK/FK	NULL	의미
<u>clientNo</u>	VARCHAR(10)	PK	No	고객No
clientName	VARCHAR(30)		Yes	이름
clientPhone	VARCHAR(13)		Yes	전화번호
clientAddress	VARCHAR(50)		Yes	주소
clientBirth	DATE		Yes	생년월일
clientHobby	VARCHAR(30)		Yes	취미
clientGener	VARCHAR(1)		Yes	성별

clientNo	clientName	clientPhone	clientAddress	clientBirth	clientHobby	clientGender
1	메시	010-1111-1111	서울	1987-06-24	축구	남
2	호날두	010-2222-2222	천안	1984-02-05		남
3	샤라포바	010-3333-3333	제주	1987-04-19		여
4	벤키	010-4444-4444	안양	1994-05-02	영화	남
5	네이마르	010-5555-5555	수원	1992-02-05	게임	남
6	윌리엄스	010-6666-6666	서울	1981-09-26	테니스	여
7	아자르	010-7777-7777	천안	1991-01-07		남
8	살라	010-8888-8888	부산	1982-04-22		남
9	루니	010-9999-9999	서울	1996-10-24	등산	남

실습 데이터베이스 준비

■ 주문테이블 bookSale

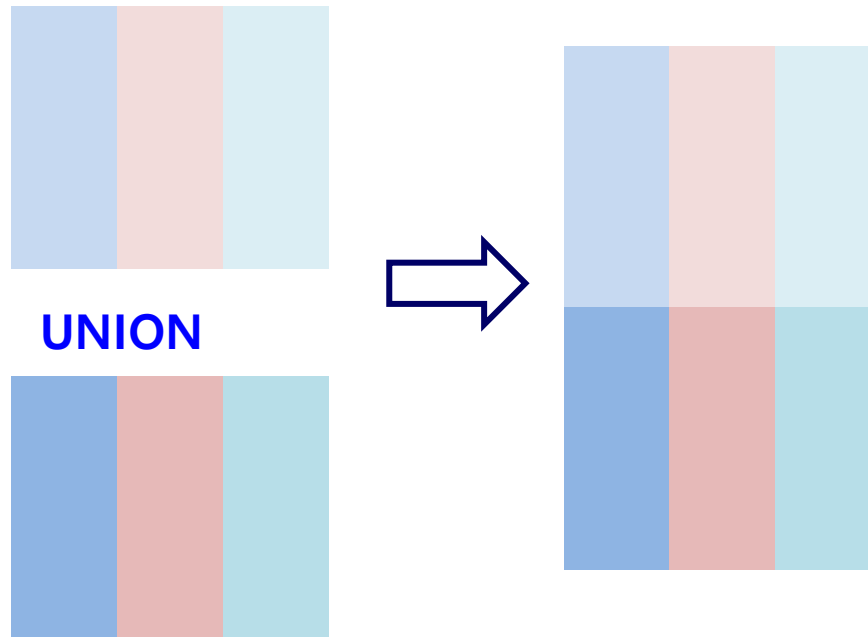
Column Name	Data type	PK/FK	NULL	의미
<u>bsNo</u>	VARCHAR(10)	PK	No	고객No
bsDate	DATE		Yes	주문일자
bsQty	INT		Yes	주문량
clientNo	VARCHAR(10)	FK	No	고객No
bookNo	VARCHAR(10)	FK	No	도서No

bsNo	bsDate	bsQty	clientNo	bookNo
1	2018-04-05	2	3	1006
2	2018-10-11	1	7	1004
3	2019-02-20	5	2	1009
4	2019-07-10	4	1	1011
5	2020-02-09	2	2	1002
6	2020-07-09	2	4	1003
7	2020-11-16	2	2	1002
8	2021-01-02	9	7	1005
9	2021-03-25	1	8	1004

유니온(UNION)

UNION 문

- 두개 이상의 테이블들을 위,아래로 결합하여 하나의 테이블로 생성
 - 조건 : 두 테이블의 컬럼들이 동일해야 함



UNION 문

■ 기본 형식

```
SELECT 열 리스트  
FROM 테이블명1  
[WHERE 조건식]
```

UNION

```
SELECT 열 리스트  
FROM 테이블명2  
[WHERE 조건식] ;
```

중복되지 않은 값으로 출력

```
SELECT 열 리스트  
FROM 테이블명1  
[WHERE 조건식]
```

UNION ALL

```
SELECT 열 리스트  
FROM 테이블명2  
[WHERE 조건식] ;
```

중복된 값 그래도 출력

UNION 문

■ 기본 형식2

```
SELECT 열 리스트  
FROM 테이블명1  
[WHERE 조건식]
```

UNION

```
SELECT 열 리스트  
FROM 테이블명2  
[WHERE 조건식]
```

UNION

```
SELECT 열 리스트  
FROM 테이블명3  
[WHERE 조건식] ;
```

테이블1과 테이블2와 테이블3을 세로
로 결합

조인(Join)

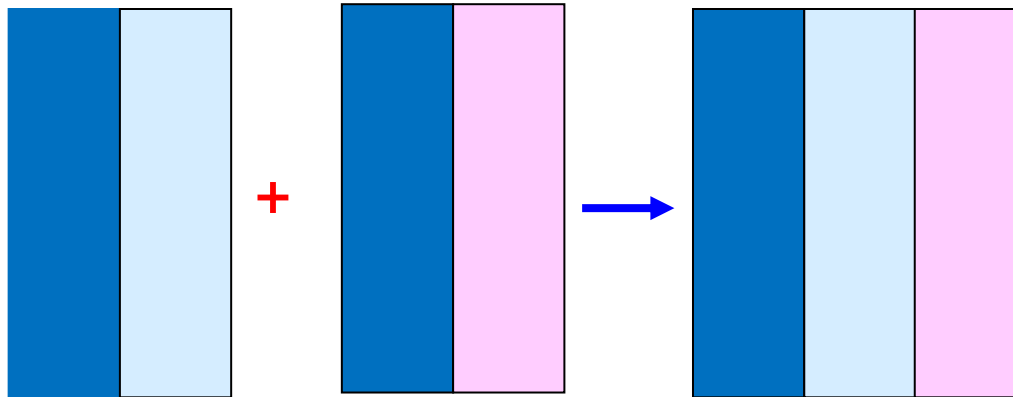
조인(Join)

- 여러 개의 테이블을 결합하여 조건에 맞는 행 검색
- 조인의 종류
 - 내부조인(Inner Join)
 - 외부조인(Outer Join)
 - 좌측 외부 조인(Left Outer Join)
 - 우측 외부 조인(Right Outer Join)
 - 완전 외부 조인(Full Outer Join)

조인(Join)

- 내부 조인(inner join)

- 공통 열이 있을 때
- 공통 열의 속성 값이 동일한 튜플(tuple)만 반환



조인(Join)

■ 내부 조인(inner join)

- 예. 2개의 테이블 내부 조인

기본키	
학과번호	학과
1	컴퓨터학과
2	경영학과
3	수학과

외래키		
학번	이름	학과번호
2022001	홍길동	1
2021002	이몽룡	2
2020003	성춘향	1
2019004	변학도	2

..... INNER JOIN



학과번호	학과	학번	이름
1	컴퓨터학과	2022001	홍길동
1	컴퓨터학과	2020003	성춘향
2	경영학과	2019004	변학도
2	경영학과	2021002	이몽룡

조인 조건 : 기본키 == 외래키

조인(Join)

■ 외부 조인(outer join)

- 공통된 속성을 매개로 하는 정보가 없더라도 버리지 않고 연산의 결과를 릴레이션에 표시
- 값이 없는 대응 속성에는 NULL 값을 채워서 반환

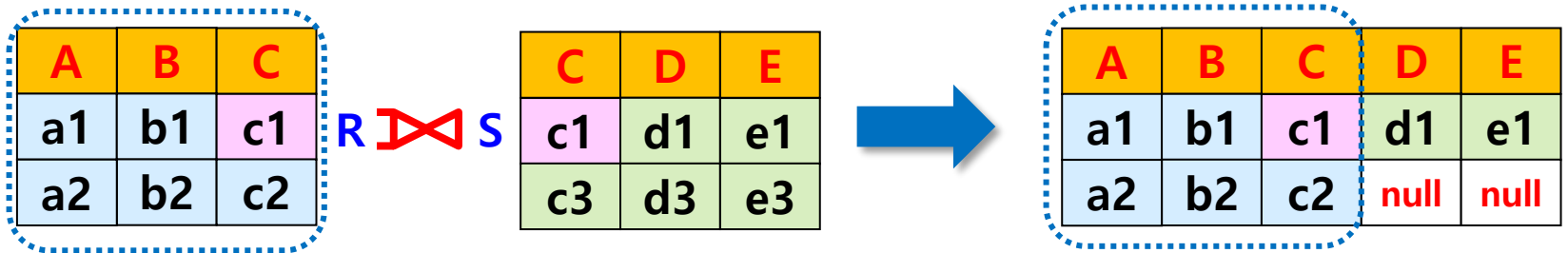
■ 외부조인 종류

- 좌측 외부 조인(Left Outer Join)
 - 좌측 릴레이션의 정보 유지
- 우측 외부 조인(Right Outer Join)
 - 우측 릴레이션의 정보 유지
- 완전 외부 조인(Full Outer Join)
 - 두 릴레이션의 모든 정보 유지

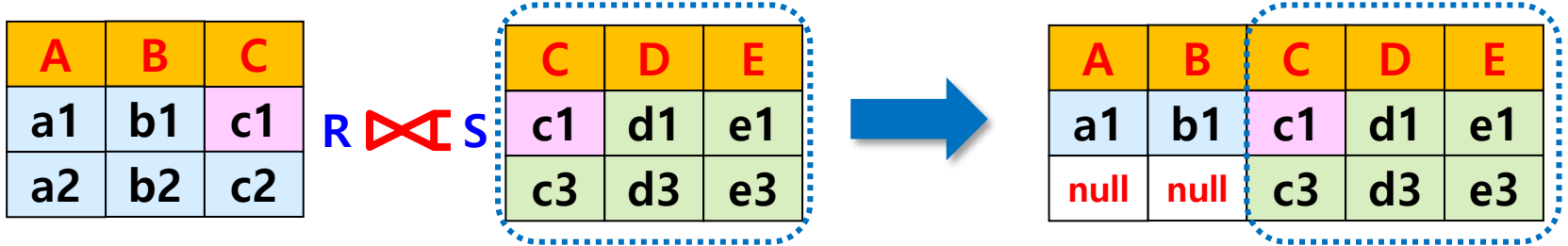
조인(Join)

■ 외부 조인(outer join)

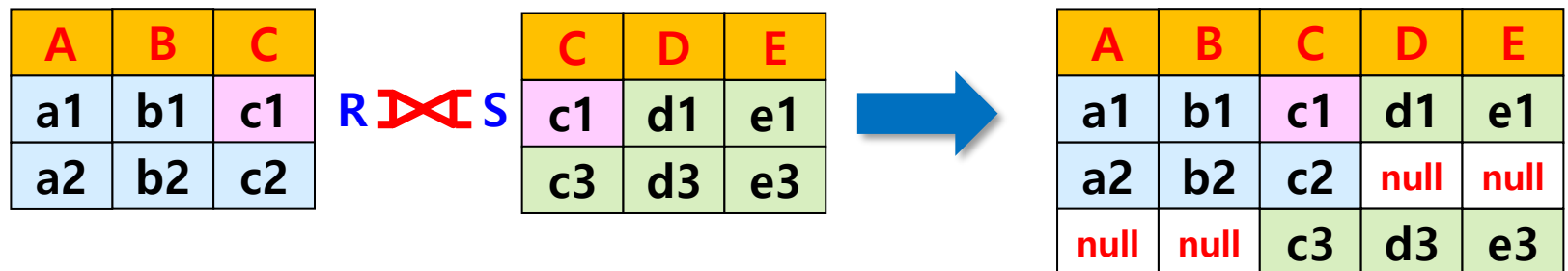
좌측 외부 조인



우측 외부 조인



완전 외부 조인



조인(Join) 문

- 내부 조인의 기본 형식

```
SELECT 열 리스트  
FROM 테이블명1, 테이블명2  
WHERE 테이블명1.키 = 테이블명2.키 ;
```

```
SELECT 열 리스트  
FROM 테이블명1  
    INNER JOIN 테이블명2  
    ON 조인조건(기본키=외래키);
```

연습문제

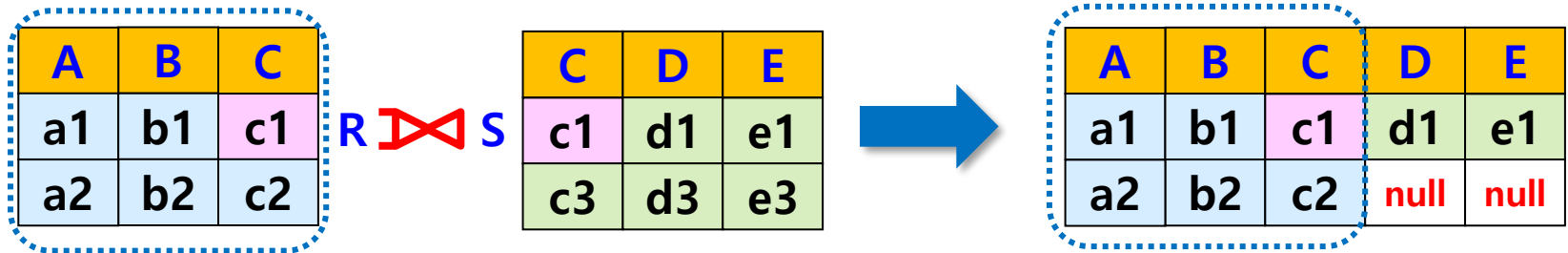
1. 모든 도서에 대하여 도서의 도서번호, 도서명, 출판사명 출력
2. '서울 출판사'에서 출간한 도서의 도서명, 저자명, 출판사명 출력 (조건에 출판사명 사용)
3. '정보출판사'에서 출간한 도서 중 판매된 도서의 도서명 출력 (중복된 경우 한 번만 출력) (조건에 출판사명 사용)
4. 도서가격이 30,000원 이상인 도서를 주문한 고객의 고객명, 도서명, 도서가격, 주문수량 출력
5. '안드로이드 프로그래밍' 도서를 구매한 고객에 대하여 도서명, 고객명, 성별, 주소 출력 (고객명으로 오름차순 정렬)
6. '도서출판 강남'에서 출간된 도서 중 판매된 도서에 대하여 '총 매출액' 출력
7. '서울 출판사'에서 출간된 도서에 대하여 판매일, 출판사명, 도서명, 도서가격, 주문수량, 주문액 출력
8. 판매된 도서에 대하여 도서별로 도서번호, 도서명, 총 주문 수량 출력
9. 판매된 도서에 대하여 고객별로 고객명, 총구매액 출력 (총구매액이 100,000원 이상인 경우만 해당)
10. 판매된 도서 중 '도서출판 강남'에서 출간한 도서에 대하여 고객명, 주문일, 도서명, 주문수량, 출판사명 출력 (고객명으로 오름차순 정렬)

외부조인(OUTER JOIN)

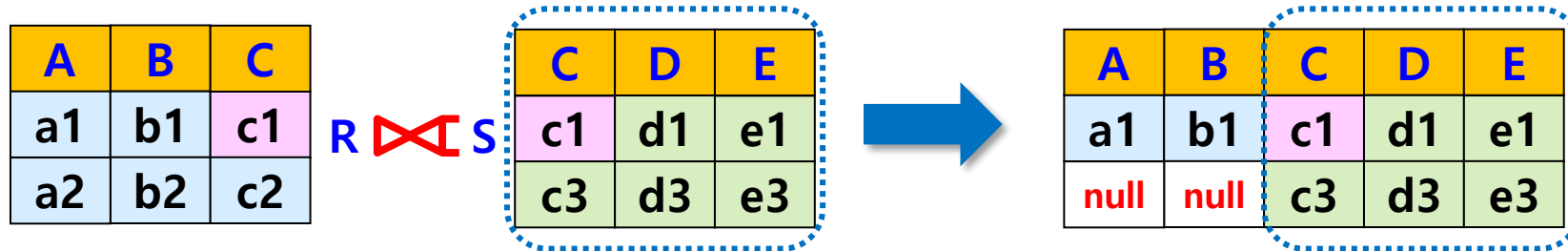
- 공통된 속성을 매개로 하는 정보가 없더라도 버리지 않고 연산의 결과를 릴레이션에 표시
- 값이 없는 대응 속성에는 NULL 값을 채워서 반환
- 좌측 외부 조인 (Left Outer JOIN)
 - 좌측 릴레이션의 정보 유지
- 우측 외부 조인 (Right Outer JOIN)
 - 우측 릴레이션의 정보 유지
- 완전 외부 조인 (Full Outer JOIN)
 - 두 릴레이션의 모든 정보 유지



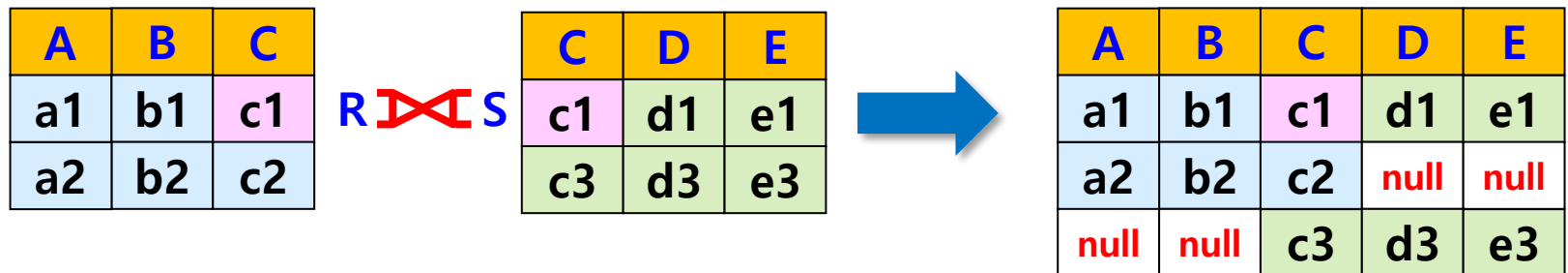
좌측 외부 조인



우측 외부 조인



완전 외부 조인



서브쿼리(Subquery)

서브쿼리(Subquery)란?

- 하위 질의 또는 부속 질의라고도 함
- 하나의 SQL 문 안에 다른 SQL 문이 중첩(nested)
- 질의를 1차 수행한 다음, 반환값을 다음 질의에 포함시켜 사용
- 다른 테이블에서 가져온 데이터로 현재 테이블에 있는 정보를 찾거나 가공할 때 사용

조인 VS 서브쿼리

■ 조인

- 여러 테이블의 데이터를 모두 합쳐서 연산
- 카티션곱($15 \times 3 = 45$ 행 반환) 후 조건에 맞는 튜플 검색
- 카티션곱 연산 + SELECT 연산

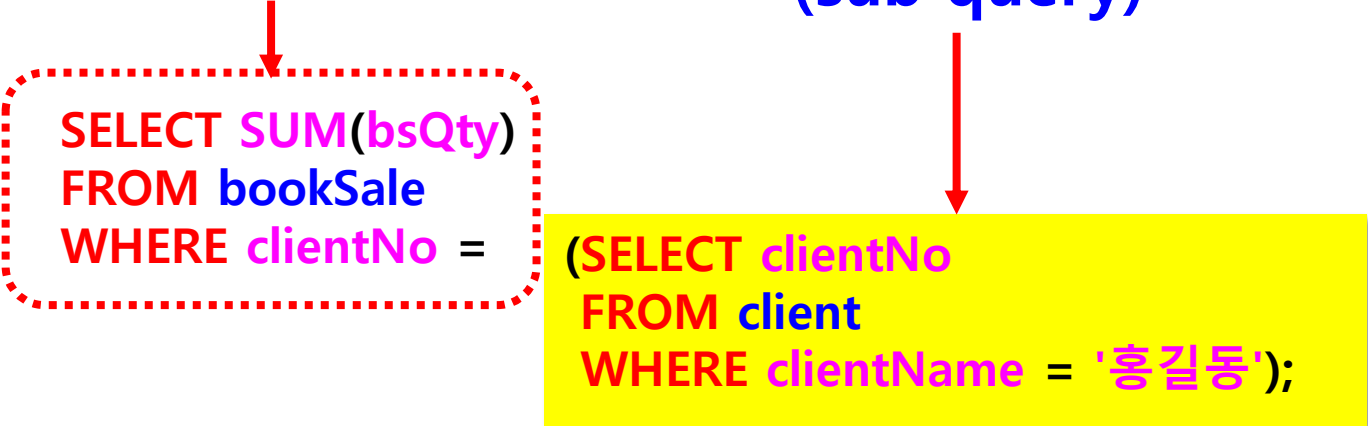
■ 서브 쿼리

- 필요한 데이터만 찾아서 제공
- 경우에 따라 조인보다 성능이 더 좋을 수도 있지만
- 대량의 데이터에서 서브 쿼리를 수행할 때 성능이 나쁠 수도 있음

서브쿼리의 구성 및 형식

- 메인 쿼리(main query)와 서브 쿼리(sub query)로 구성

(main query)



```
SELECT SUM(bsQty)
FROM bookSale
WHERE clientNo =
```

(sub query)

```
(SELECT clientNo
FROM client
WHERE clientName = '홍길동');
```

서브쿼리 결과 값에 따른 구분

■ 단일 행 서브쿼리

- 서브쿼리 결과 값이 단일 행
- = 연산자 사용

■ 다중 행 서브쿼리

- 서브쿼리 결과 값이 여러 행
- IN, ANY, ALL, EXISTS 연산자 사용

서브쿼리 결과 값에 따른 구분

■ 서브쿼리 연산자

- WHERE 절에서 사용
- 데이터를 선택하는 조건 또는 술어(predicate)와 같이 사용

연산	연산자	반환 행
비교	=, >, <, >=, <=, !=	단일
집합	IN, NOT IN	다중
존재	EXISTS, NOT EXISTS	다중
한정	ALL (모두), ANY (최소 하나라도)	다중

참고. IN과 EXISTS의 차이

■ IN

- 부속 질의에서 조건에 해당되는 ROW의 열을 비교하여 **값 확인**
- 서브 쿼리의 결과 값을 메인 쿼리에 대입하여 조건 비교 후 결과 출력
- IN 쿼리 → 메인 쿼리

②

```
SELECT SUM(bsQty) AS "총 판매량"  
FROM bookSale  
WHERE clientNo IN (SELECT clientNo  
FROM client  
WHERE clientAddress LIKE '%인천%');
```

①

참고. IN과 EXISTS의 차이

■ EXISTS

- 부속 질의에서 **조건에 해당되는 ROW의 존재 여부만 확인** (TRUE/FALSE 반환)
- 따라서 IN에 비해 성능 좋음
- EXISTS 키워드 앞에 속성명, 수식 등이 올 수 없음
- WHERE 절에 외래키 제약조건 지정
- NULL 값 포함 : NULL 값도 결과에 포함
 - IN : NULL 값이 결과에 포함되지 않음

참고. NOT EXISTS와 NOT IN 차이

- 대부분 NOT EXISTS와 NOT IN 결과 동일하지만
- 공통되는 조인 속성의 값이 NULL인 경우 결과 다름
 - NOT EXISTS : NULL 값도 결과에 포함
 - NOT IN : NULL 값이 결과에 포함되지 않음

다중 행 서브쿼리

■ ALL / ANY, SOME : 관계 연산자 뒤에 위치

■ ALL

- 검색 조건이 서브 쿼리의 결과의 모든 값에 만족하면 참이 되는 연산자
- 조건 > ALL (서브 쿼리 결과)

■ ANY, SOME

- 검색 조건이 서브 쿼리의 결과 중에서 하나 이상에 만족하면 참이 되는 연산자
- 조건 > ANY (서브 쿼리 결과)

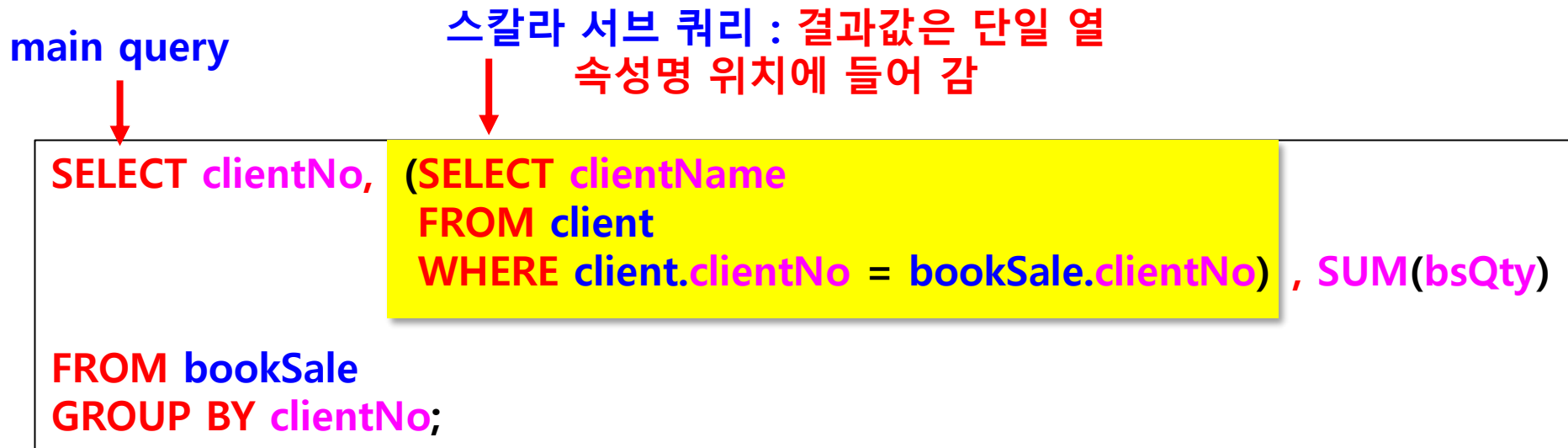
서브쿼리 유형

명칭	사용 위치	영문 및 동의어	설명
스칼라 서브 쿼리	SELECT 절	Scalar Subquery 열이름 위치에 표기	단일 열 반환
인라인 뷰	FROM 절	Inline View Table Subquery 테이블명 위치에 표기	결과를 뷰(View) 형태로 반환 (가상 테이블)
중첩 서브 쿼리	WHERE 절	Nested Subquery Predicate Subquery	술어와 같이 사용 결과를 한정시키기 위해 사용

서브쿼리 유형

■ 스칼라 서브쿼리(Scalar Subquery)

- SELECT 절에서 사용
- 결과 값을 단일 열의 스칼라 값으로 반환
- 스칼라 값이 들어갈 수 있는 모든 곳에서 사용 가능
- 일반적으로 SELECT 문과 UPDATE SET 문에서 사용



MySQL 내장 함수

내장 함수

- 문자열 함수
- 날짜 함수
- 집계 함수
- 수학 함수
- 순위 함수
- 분석 함수

문자열 함수

- 문자열 연결 : CONCAT, CONCAT_WS
- 문자열 일부 추출 : SUBSTR, SUBSTRING
- 형변환 : CAST, CONVERT
- 치환 : REPLACE
- 문자열 길이 : CHAR_LENGTH, LENGTH
- 대소문자 변환 : UPPER, LOWER
- 공백 제거 : TRIM
- NULL 대체 : IFNULL, COALESE

날짜 함수

- 날짜나 시간 데이터를 활용하기 위한 함수
- 일정 기간의 데이터를 검색할 때 자주 사용
- 서버의 현재 날짜나 시간
 - CURRENT_DATE, CURRENT_TIME, CURRENT_TIMESTAMP, NOW

■ 서버의 현재 날짜나 시간

- `CURRENT_DATE()`
 - 서버의 현재 날짜 반환
- `CURRENT_TIME()`
 - 서버의 현재 시간 반환
 - 3을 인수로 주는 경우 밀리초 단위까지 확인
- `CURRENT_TIMESTAMP()`, `NOW()`
 - 서버의 현재 날짜와 시간 반환
 - 3을 인수로 주는 경우 밀리초 단위까지 확인
- `UTC_DATE()`
 - 세계 표준 날짜 반환
- `UTC_TIME()`
 - 세계 표준 시간 반환, 3을 인수로 주는 경우 밀리초 단위까지 확인
- `UTC_TIMESTAMP()`
 - 세계 표준 날짜와 시간 반환, 3을 인수로 주는 경우 밀리초 단위까지 확인

■ ROUND 함수

- 반올림한 값을 구하는 함수
- ROUND(값, 자리수)
 - 자리수 아래서 반올림하여 자리수까지 출력
 - 양수 값 : 소수점 오른쪽 자릿수
 - 음수 값 : 소수점 왼쪽 자릿수
- 예: 소수 첫째 자리까지 반올림한 값 출력
 - `SELECT ROUND(3.862, 1);` -- 3.900
- 소수 둘째 자리까지 반올림한 값 출력
 - `SELECT ROUND(3.862, 2);` -- 3.860

- 순위 출력 : RANK(), DENSE_RANK(), ROW_NUMBER()
 - RANK()
 - 값의 순위 반환
 - 동일 순위 개수만큼 증가
 - DENSE_RANK()
 - 값의 순위 반환
 - 동일 순위 상관 없이 1 증가
 - ROW_NUMBER()
 - 행의 순위 반환